

Supplementary Material for REAP: Learning General Rearrangement Heuristics from Motion-Planner Feedback

Anonymous Author(s)

Affiliation

Address

email

1 Overview

This document supplements the main paper with the full PDDL domain definitions used during training and evaluation of REAP across the five rearrangement environments described in Sections 4.1 and 4.3 of the main paper. For each environment we include the domain file consumed by the relational graph neural network heuristic at training time, with the same predicate set, action schemas, and derived-predicate structure used to produce the reported results.

Per-environment PDDL summary. Three of the five environments use plain STRIPS domains; the remaining two use derived predicates that surface a structural occlusion (resp. support) relation to the GNN without changing the underlying action semantics.

Main-paper Env. name	Source file	Variant
Open-Tabletop Rearrangement	open_tabletop.pddl	STRIPS
Confined-Shelf Rearrangement	confined_shelf.pddl	STRIPS
Multi-Level Shelf Retrieval	multilevel_shelf.pddl	STRIPS
Access-19	access19.pddl	+ (blocking) derived
3D Block Stacking	blocks.pddl	+ (supported) derived

The remainder of this document reproduces each domain file verbatim.

2 PDDL Domain Definitions

2.1 Open-Tabletop Rearrangement

```
(define (domain tabletop-manipulation)
  (:requirements :strips :typing :negative-preconditions :derived-predicates
    ↪ :disjunctive-preconditions :existential-preconditions)

  (:types
    cell cylinder direction
  )

  (:constants
    north east - direction
  )

  (:predicates
    (adjacent ?dir - direction ?cel1 - cell ?cel2 - cell) ; cell cel2 is
    ↪ adjacent to cel1 in direction dir
    (occupied ?cel - cell ?cyl - cylinder) ; cell contains cylinder
    (empty ?cel - cell) ; cell is empty
```

```

16      (holding ?cyl - cylinder)          ; gripper is holding cylinder cyl
17      (gripper-empty)                   ; gripper is not holding anything
18      (discarded ?cyl - cylinder)       ; cylinder is discarded
19  )
20
21  (:action pick
22    :parameters (?cyl - cylinder ?cel - cell)
23    :precondition (and
24      (gripper-empty)
25      (occupied ?cel ?cyl)
26    )
27    :effect (and
28      (not (gripper-empty))
29      (not (occupied ?cel ?cyl))
30      (empty ?cel)
31      (holding ?cyl)
32    )
33  )
34
35  (:action put
36    :parameters (?cyl - cylinder ?cel - cell)
37    :precondition (and
38      (holding ?cyl)
39      (empty ?cel)
40    )
41    :effect (and
42      (gripper-empty)
43      (occupied ?cel ?cyl)
44      (not (empty ?cel))
45      (not (holding ?cyl))
46    )
47  )
48 )

```

14 2.2 Confined-Shelf Rearrangement

```

1  ;; Confined-shelf rearrangement domain (Wang ICAPS-2022).
2  ;;
3  ;; Reuses the tabletop predicate vocabulary (occupied / empty / holding /
4  ;; gripper-empty) so rgnet's existing predicate handling carries over.
5  ;; Adds:
6  ;;   - 'direction' type with north/east constants + (adjacent ?dir ?c1 ?c2)
7  ;;     static grid relation -- the structural signal the GNN propagates
8  ;;     embeddings along; pick/put preconditions don't use it,
9  ;;   - 'color' type + (color-of ?cyl ?c) static predicate for the
10 ;;     colour-by-column goal.
11 ;;
12 ;; Action semantics match the tabletop 'spatial_put' variant: pick removes
13 ;; an object from a cell into the gripper; put places the held object at a
14 ;; specific empty cell. No drop/discard.
15 ;;
16 ;; The confined-shelf nature of the problem -- front-occlusion and lateral
17 ;; gripper-clearance BLOCKING -- is NOT encoded symbolically; PDDL only
18 ;; knows occupied/empty (can't pick an empty cell, can't put to an occupied
19 ;; one). Feasibility (which cylinder blocks which) lives entirely in the
20 ;; external feasibility checker the rearrangement search consults.
21
22 (define (domain confined-shelf)
23   (:requirements :strips :typing :negative-preconditions)
24
25   (:types
26     cell cylinder color direction
27   )

```

```

28
29 (:constants
30   north east - direction
31 )
32
33 (:predicates
34   (adjacent ?dir - direction ?cel1 - cell ?cel2 - cell) ; static grid edge
35   (occupied ?cel - cell ?cyl - cylinder) ; cell holds the cylinder
36   (empty ?cel - cell) ; cell is unoccupied
37   (holding ?cyl - cylinder) ; gripper currently holds cyl
38   (gripper-empty) ; gripper not holding anything
39   (color-of ?cyl - cylinder ?c - color) ; static -- cylinder colour group
40 )
41
42 (:action pick
43   :parameters (?cyl - cylinder ?cel - cell)
44   :precondition (and
45     (gripper-empty)
46     (occupied ?cel ?cyl)
47   )
48   :effect (and
49     (not (gripper-empty))
50     (not (occupied ?cel ?cyl))
51     (empty ?cel)
52     (holding ?cyl)
53   )
54 )
55
56 (:action put
57   :parameters (?cyl - cylinder ?cel - cell)
58   :precondition (and
59     (holding ?cyl)
60     (empty ?cel)
61   )
62   :effect (and
63     (gripper-empty)
64     (not (holding ?cyl))
65     (not (empty ?cel))
66     (occupied ?cel ?cyl)
67   )
68 )
69 )

```

15 2.3 Multi-Level Shelf Retrieval

```

1 ;; Tabletop-access domain -- filter-mode (Bouhsain HAL 2025).
2 ;;
3 ;; Reuses the confined-shelf / access-19 predicate vocabulary
4 ;; (occupied / empty / holding / gripper-empty) and adds the structural
5 ;; ``adjacent ?dir ?c1 ?c2`` relation with a ``direction`` type and
6 ;; ``north``/``east`` constants. Pick/put preconditions DON'T use
7 ;; adjacency -- it's the spatial grid signal for the GNN, whose message
8 ;; passing propagates embeddings along these edges to learn the
9 ;; workspace's spatial semantics (front-to-back row order, column
10 ;; adjacency) rather than parsing it from cell-id strings.
11 ;;
12 ;; The ``access`` scene has FOUR disjoint grids -- floor_left, floor_right,
13 ;; middle_deck, top_deck. Adjacency is emitted WITHIN each grid in the
14 ;; problem file (north = +iy depth, east = +ix column); there is no
15 ;; cross-grid adjacency (levels are traversed by the chain executor, not
16 ;; the symbolic layer).
17 ;;
18 ;; In filter mode, grasp face is a refinement choice handled by the

```

```

19 ;; feasibility checker; PDDL only sees ``(pick ?obj ?cel)`` and
20 ;; ``(put ?obj ?cel)``. See ``domain_face.pddl`` for the parameterised
21 ;; variant.
22
23 (define (domain tabletop-access)
24   (:requirements :strips :typing :negative-preconditions)
25
26   (:types
27     cell movable direction
28   )
29
30   (:constants
31     north east - direction
32   )
33
34   (:predicates
35     (adjacent ?dir - direction ?cel1 - cell ?cel2 - cell)
36     (occupied ?cel - cell ?obj - movable)
37     (empty ?cel - cell)
38     (holding ?obj - movable)
39     (gripper-empty)
40   )
41
42   (:action pick
43     :parameters (?obj - movable ?cel - cell)
44     :precondition (and
45       (gripper-empty)
46       (occupied ?cel ?obj)
47     )
48     :effect (and
49       (not (gripper-empty))
50       (not (occupied ?cel ?obj))
51       (empty ?cel)
52       (holding ?obj)
53     )
54   )
55
56   (:action put
57     :parameters (?obj - movable ?cel - cell)
58     :precondition (and
59       (holding ?obj)
60       (empty ?cel)
61     )
62     :effect (and
63       (gripper-empty)
64       (not (holding ?obj))
65       (not (empty ?cel))
66       (occupied ?cel ?obj)
67     )
68   )
69 )

```

16 2.4 Access-19

```

1  ;; Tabletop-access domain -- rgnet/GNN dataset variant with derived
2  ;; predicates. Identical to ``domain.pddl`` except for:
3  ;;
4  ;; * ``:derived-predicates`` requirement
5  ;; * new ``(blocking ?obj ?goal-cell)`` derived predicate
6  ;;
7  ;; The derived rule says: an object ``?o`` is blocking goal cell
8  ;; ``?goal`` iff it currently occupies a cell in the SAME cube column
9  ;; AND closer to the open -y face than ``?goal``. Reads

```

```

10 ;; `(same-column-front ...)` static facts emitted per problem (same
11 ;; facts both domain variants see).
12 ;;
13 ;; Why a separate file: unified-planning rejects
14 ;; `:derived-predicates`; the planner runtime uses `domain.pddl`
15 ;; (without the requirement). The rgnet dataset bundle includes
16 ;; BOTH files -- rgnet loads this one so its GNN sees the blocking
17 ;; structural signal. Problem files are unchanged regardless of
18 ;; which domain is loaded (the static `(same-column-front ...)`
19 ;; facts are declared in both domains; `(blocking ...)` is purely
20 ;; derived here, never appears in problem `:init`).
21
22 (define (domain tabletop-access)
23   (:requirements :strips :typing :negative-preconditions
24                 :disjunctive-preconditions :existential-preconditions
25                 :derived-predicates)
26
27   (:types
28     cell movable direction
29   )
30
31   (:constants
32     north east - direction
33   )
34
35   (:predicates
36     (adjacent ?dir - direction ?cel1 - cell ?cel2 - cell)
37     (occupied ?cel - cell ?obj - movable)
38     (empty ?cel - cell)
39     (holding ?obj - movable)
40     (gripper-empty)
41     (same-column-front ?cel-front - cell ?cel-back - cell)
42     ;; Derived: `?o` is occupying a cell that is in the same cube
43     ;; column as `?goal` and strictly in front of it (smaller iy).
44     ;; True iff there exists such a cell.
45     (blocking ?obj - movable ?goal - cell)
46   )
47
48   (:derived (blocking ?obj - movable ?goal - cell)
49     (exists (?c - cell)
50       (and (occupied ?c ?obj)
51         (same-column-front ?c ?goal))))
52
53   (:action pick
54     :parameters (?obj - movable ?cel - cell)
55     :precondition (and
56       (gripper-empty)
57       (occupied ?cel ?obj)
58     )
59     :effect (and
60       (not (gripper-empty))
61       (not (occupied ?cel ?obj))
62       (empty ?cel)
63       (holding ?obj)
64     )
65   )
66
67   (:action put
68     :parameters (?obj - movable ?cel - cell)
69     :precondition (and
70       (holding ?obj)
71       (empty ?cel)
72     )
73     :effect (and
74       (gripper-empty)

```

```

75     (not (holding ?obj))
76     (not (empty ?cel))
77     (occupied ?cel ?obj)
78   )
79 )
80 )

```

17 2.5 Three-Dimensional Block Stacking

```

1  ;; Multi-level wooden-blocks domain (Kulshrestha CoRL-2023 -- redesigned 2026-05).
2  ;;
3  ;; Two-table workspace:
4  ;;
5  ;; * ``parts`` -- 2D :class:`GridRegion` behind the robot. Blocks rest on
6  ;;   this surface; no stacking here.
7  ;; * ``stack_L0`` ... ``stack_L4`` -- five 2D regions on the stack table in
8  ;;   front of the robot, one per vertical level. The 3D stack grid lives
9  ;;   here. Cells in ``stack_L0`` are at table-top z; ``stack_L<k>`` cells
10 ;;   sit one cube-size higher than ``stack_L<k-1>``.
11 ;;
12 ;; Block shapes:
13 ;;
14 ;; * 1x1 cube (``(cube ?b)``): occupies one cell.
15 ;; * 2x1 oblong (``(oblong ?b)``): occupies two cells. Three valid
16 ;;   orientations:
17 ;;   - flat-x: two cells along +x in the same level (``east-of c1 c2``)
18 ;;   - flat-y: two cells along +y in the same level (``north-of c1 c2``)
19 ;;   - upright: two cells at the same (ix,iy) one level apart
20 ;;     (``above c_low c_high``). Upright only valid on the stack table.
21 ;;
22 ;; Grasps:
23 ;;
24 ;; * pick-cube: top-down, centred.
25 ;; * pick-flat-x / pick-flat-y: top-down on the long sides of the oblong.
26 ;; * pick-upright: front-facing horizontal grasp on the vertical sides.
27 ;;
28 ;; In-hand transforms (no MP collision risk -- performed in free air):
29 ;;
30 ;; * make-upright-from-{x,y}: tilt the held block from horizontal to vertical.
31 ;; * make-flat-{x,y}-from-upright: reverse tilt.
32 ;; * turn-x-to-y / turn-y-to-x: yaw 90 deg in the horizontal plane.
33
34 (define (domain multilevel-blocks)
35   (:requirements
36     :strips :typing :negative-preconditions
37     :disjunctive-preconditions :existential-preconditions
38     :derived-predicates
39   )
40
41   (:types
42     cell block
43   )
44
45   (:predicates
46     ;; --- static (set in initial state of every problem) ---
47     (cube ?b - block)
48     (oblong ?b - block)
49     (long ?b - block)
50
51     ; 3x1 long block -- same
52     ; shape family as oblong but
53     ; 3 cells along the long axis;
54     ; support rule is (centre
55     ; alone) OR (both ends), so
56     ; only single-end-cantilever

```

```

55                                     ; layouts are rejected (vs
56                                     ; oblong's "every cell must
57                                     ; be supported").
58 (in-parts ?c - cell)
59 (in-stack ?c - cell)
60 ;; Static: ``puttable`` is True for every cell that can be a put-* target.
61 ;; In Phase 1 of the multilevel_blocks redesign this is set to be True
62 ;; for every ``stack_L*`` cell and False (i.e., not asserted) for every
63 ;; ``parts_*`` cell -- the parts table is now strictly a pick source.
64 ;; Eliminates the action-space bloat of ``put-* ?b parts__X_Y`` actions
65 ;; from rgnet's GNN expansion at every state.
66 (puttable ?c - cell)
67 ;; directional adjacency -- asymmetric. 3 predicates cover all 6
68 ;; geometric directions: the "reverse" direction is the same predicate
69 ;; with parameters swapped.
70 (above ?c-low - cell ?c-up - cell)           ; +z
71 (east-of ?c1 - cell ?c2 - cell)              ; +x; c2 is east of c1
72 (north-of ?c1 - cell ?c2 - cell)             ; +y; c2 is north of c1
73
74 ;; --- fluent ---
75 (in ?b - block ?c - cell)                    ; block occupies cell (multi-
76                                           ; cell blocks have multiple facts)
77 (empty ?c - cell)
78 (gripper-empty)
79 (held-cube ?b - block)
80 (held-flat-x ?b - block)
81 (held-flat-y ?b - block)
82 (held-upright ?b - block)
83
84 ;; --- derived ---
85 ;; Cell ?c is supported: nothing directly below it (table-level), OR
86 ;; the cell directly below is non-empty. Replaces the inline
87 ;; (or (not (exists ...)) (exists ...)) support-check that used to live
88 ;; in every put-* precondition. Inline disjunctive action preconditions
89 ;; make pymimir split each put-* into multiple action schemas with
90 ;; witness parameters of differing arity, which breaks the GNN encoder's
91 ;; one-update-module-per-name assumption. As a derived predicate the
92 ;; existential stays out of the action schemas, so each put-* keeps a
93 ;; single arity. (puttable is kept as-is -- it's the static put-target
94 ;; pre-filter and does not introduce witness parameters.)
95 (supported ?c - cell)
96 )
97
98 (:derived (supported ?c - cell)
99   (or
100     (not (exists (?cb - cell) (above ?cb ?c)))
101     (exists (?cb - cell)
102       (and (above ?cb ?c) (not (empty ?cb))))))
103
104 ;; =====
105 ;; PICK actions
106 ;; =====
107
108 ;; Pick a cube. The cell directly above (if any) must be empty so the
109 ;; cube being picked isn't supporting another block.
110 (:action pick-cube
111   :parameters (?b - block ?c - cell)
112   :precondition (and
113     (cube ?b) (in ?b ?c) (gripper-empty)
114     (not (exists (?c-above - cell)
115       (and (above ?c ?c-above) (not (empty ?c-above))))))
116   )
117   :effect (and
118     (not (in ?b ?c)) (empty ?c)
119     (held-cube ?b) (not (gripper-empty))

```

```

120 )
121 )
122
123 ;; Pick a flat-x oblong (block spans c1, c2 with c2 east of c1).
124 (:action pick-flat-x
125   :parameters (?b - block ?c1 - cell ?c2 - cell)
126   :precondition (and
127     (oblong ?b) (in ?b ?c1) (in ?b ?c2) (east-of ?c1 ?c2)
128     (gripper-empty)
129     (not (exists (?ca - cell)
130       (and (above ?c1 ?ca) (not (empty ?ca))))))
131     (not (exists (?ca - cell)
132       (and (above ?c2 ?ca) (not (empty ?ca))))))
133   )
134   :effect (and
135     (not (in ?b ?c1)) (not (in ?b ?c2))
136     (empty ?c1) (empty ?c2)
137     (held-flat-x ?b) (not (gripper-empty))
138   )
139 )
140
141 ;; Pick a flat-y oblong (block spans c1, c2 with c2 north of c1).
142 (:action pick-flat-y
143   :parameters (?b - block ?c1 - cell ?c2 - cell)
144   :precondition (and
145     (oblong ?b) (in ?b ?c1) (in ?b ?c2) (north-of ?c1 ?c2)
146     (gripper-empty)
147     (not (exists (?ca - cell)
148       (and (above ?c1 ?ca) (not (empty ?ca))))))
149     (not (exists (?ca - cell)
150       (and (above ?c2 ?ca) (not (empty ?ca))))))
151   )
152   :effect (and
153     (not (in ?b ?c1)) (not (in ?b ?c2))
154     (empty ?c1) (empty ?c2)
155     (held-flat-y ?b) (not (gripper-empty))
156   )
157 )
158
159 ;; Pick an upright oblong (block spans c-low, c-high with c-high above c-low).
160 ;; Only valid on the stack table. The cell directly above c-high (if any)
161 ;; must be empty.
162 (:action pick-upright
163   :parameters (?b - block ?c-low - cell ?c-high - cell)
164   :precondition (and
165     (oblong ?b) (in ?b ?c-low) (in ?b ?c-high) (above ?c-low ?c-high)
166     (gripper-empty)
167     (not (exists (?ca - cell)
168       (and (above ?c-high ?ca) (not (empty ?ca))))))
169   )
170   :effect (and
171     (not (in ?b ?c-low)) (not (in ?b ?c-high))
172     (empty ?c-low) (empty ?c-high)
173     (held-upright ?b) (not (gripper-empty))
174   )
175 )
176
177 ;; =====
178 ;; PUT actions
179 ;; =====
180
181 ;; Put a cube. Cell c must be empty and EITHER at table-level (no cell
182 ;; below) OR directly above an occupied cell.
183 (:action put-cube
184   :parameters (?b - block ?c - cell)

```



```

185 :precondition (and
186   (cube ?b) (held-cube ?b) (empty ?c)
187   (puttable ?c)
188   (supported ?c)
189 )
190 :effect (and
191   (in ?b ?c) (not (empty ?c))
192   (not (held-cube ?b)) (gripper-empty)
193 )
194 )
195
196 ;; Put a held-flat-x oblong at c1, c2 (c2 east of c1, same level).
197 ;; Both cells must be empty and supported (table-level OR cell below
198 ;; occupied).
199 (:action put-flat-x
200   :parameters (?b - block ?c1 - cell ?c2 - cell)
201   :precondition (and
202     (oblong ?b) (held-flat-x ?b) (east-of ?c1 ?c2)
203     (empty ?c1) (empty ?c2)
204     (puttable ?c1) (puttable ?c2)
205     (supported ?c1) (supported ?c2)
206   )
207   :effect (and
208     (in ?b ?c1) (in ?b ?c2)
209     (not (empty ?c1)) (not (empty ?c2))
210     (not (held-flat-x ?b)) (gripper-empty)
211   )
212 )
213
214 ;; Put a held-flat-y oblong at c1, c2 (c2 north of c1, same level).
215 (:action put-flat-y
216   :parameters (?b - block ?c1 - cell ?c2 - cell)
217   :precondition (and
218     (oblong ?b) (held-flat-y ?b) (north-of ?c1 ?c2)
219     (empty ?c1) (empty ?c2)
220     (puttable ?c1) (puttable ?c2)
221     (supported ?c1) (supported ?c2)
222   )
223   :effect (and
224     (in ?b ?c1) (in ?b ?c2)
225     (not (empty ?c1)) (not (empty ?c2))
226     (not (held-flat-y ?b)) (gripper-empty)
227   )
228 )
229
230 ;; Put a held-upright oblong at c-low, c-high (c-high above c-low).
231 ;; Only c-low needs the support check; c-high inherits support from
232 ;; being directly above c-low (which the block itself will occupy).
233 (:action put-upright
234   :parameters (?b - block ?c-low - cell ?c-high - cell)
235   :precondition (and
236     (oblong ?b) (held-upright ?b) (above ?c-low ?c-high)
237     (empty ?c-low) (empty ?c-high)
238     (puttable ?c-low) (puttable ?c-high)
239     (supported ?c-low)
240   )
241   :effect (and
242     (in ?b ?c-low) (in ?b ?c-high)
243     (not (empty ?c-low)) (not (empty ?c-high))
244     (not (held-upright ?b)) (gripper-empty)
245   )
246 )
247
248 ;; =====
249 ;; In-hand TRANSFORM actions (free-air gripper rotation)

```

```

250 ;; =====
251 ;; No MP collision check needed -- gripper rotates while held above the
252 ;; workspace. Bridge feasibility verifies the rotation actually clears
253 ;; the air space (e.g., the held block doesn't sweep through anything).
254
255 (:action make-upright-from-x
256   :parameters (?b - block)
257   :precondition (and (oblong ?b) (held-flat-x ?b))
258   :effect (and (not (held-flat-x ?b)) (held-upright ?b))
259 )
260
261 (:action make-upright-from-y
262   :parameters (?b - block)
263   :precondition (and (oblong ?b) (held-flat-y ?b))
264   :effect (and (not (held-flat-y ?b)) (held-upright ?b))
265 )
266
267 (:action make-flat-x-from-upright
268   :parameters (?b - block)
269   :precondition (and (oblong ?b) (held-upright ?b))
270   :effect (and (not (held-upright ?b)) (held-flat-x ?b))
271 )
272
273 (:action make-flat-y-from-upright
274   :parameters (?b - block)
275   :precondition (and (oblong ?b) (held-upright ?b))
276   :effect (and (not (held-upright ?b)) (held-flat-y ?b))
277 )
278
279 (:action turn-x-to-y
280   :parameters (?b - block)
281   :precondition (and (oblong ?b) (held-flat-x ?b))
282   :effect (and (not (held-flat-x ?b)) (held-flat-y ?b))
283 )
284
285 (:action turn-y-to-x
286   :parameters (?b - block)
287   :precondition (and (oblong ?b) (held-flat-y ?b))
288   :effect (and (not (held-flat-y ?b)) (held-flat-x ?b))
289 )
290
291 ;; =====
292 ;; 3x1 LONG block actions
293 ;; =====
294 ;; The long block reuses the held-flat-{x,y} / held-upright fluents;
295 ;; the (long ?b) static predicate selects between the oblong and long
296 ;; action variants. Pick/put preconditions span 3 cells. Put-long-{x,y}
297 ;; uses a permissive support rule: the CENTRE alone suffices (the long
298 ;; balances on its midpoint) OR both ENDS together suffice (the long
299 ;; bridges across an empty centre). Only single-end-cantilever layouts
300 ;; are rejected.
301
302 (:action pick-long-x
303   :parameters (?b - block ?c1 - cell ?c2 - cell ?c3 - cell)
304   :precondition (and
305     (long ?b) (in ?b ?c1) (in ?b ?c2) (in ?b ?c3)
306     (east-of ?c1 ?c2) (east-of ?c2 ?c3)
307     (gripper-empty)
308     (not (exists (?ca - cell)
309       (and (above ?c1 ?ca) (not (empty ?ca))))))
310     (not (exists (?ca - cell)
311       (and (above ?c2 ?ca) (not (empty ?ca))))))
312     (not (exists (?ca - cell)
313       (and (above ?c3 ?ca) (not (empty ?ca))))))
314 )

```

```

315 :effect (and
316   (not (in ?b ?c1)) (not (in ?b ?c2)) (not (in ?b ?c3))
317   (empty ?c1) (empty ?c2) (empty ?c3)
318   (held-flat-x ?b) (not (gripper-empty))
319 )
320 )
321
322 (:action pick-long-y
323 :parameters (?b - block ?c1 - cell ?c2 - cell ?c3 - cell)
324 :precondition (and
325   (long ?b) (in ?b ?c1) (in ?b ?c2) (in ?b ?c3)
326   (north-of ?c1 ?c2) (north-of ?c2 ?c3)
327   (gripper-empty)
328   (not (exists (?ca - cell)
329     (and (above ?c1 ?ca) (not (empty ?ca)))))
330   (not (exists (?ca - cell)
331     (and (above ?c2 ?ca) (not (empty ?ca)))))
332   (not (exists (?ca - cell)
333     (and (above ?c3 ?ca) (not (empty ?ca)))))
334 )
335 :effect (and
336   (not (in ?b ?c1)) (not (in ?b ?c2)) (not (in ?b ?c3))
337   (empty ?c1) (empty ?c2) (empty ?c3)
338   (held-flat-y ?b) (not (gripper-empty))
339 )
340 )
341
342 (:action pick-long-upright
343 :parameters (?b - block ?c-low - cell ?c-mid - cell ?c-high - cell)
344 :precondition (and
345   (long ?b) (in ?b ?c-low) (in ?b ?c-mid) (in ?b ?c-high)
346   (above ?c-low ?c-mid) (above ?c-mid ?c-high)
347   (gripper-empty)
348   (not (exists (?ca - cell)
349     (and (above ?c-high ?ca) (not (empty ?ca)))))
350 )
351 :effect (and
352   (not (in ?b ?c-low)) (not (in ?b ?c-mid)) (not (in ?b ?c-high))
353   (empty ?c-low) (empty ?c-mid) (empty ?c-high)
354   (held-upright ?b) (not (gripper-empty))
355 )
356 )
357
358 ;; Put a held flat-x long block at c1, c2, c3 (east-of chain). Each
359 ;; of c1, c2, c3 must be empty. Support rule: the CENTRE cell c2
360 ;; supported alone suffices (the long balances on its midpoint), OR
361 ;; both ENDS c1 and c3 supported (the long bridges across an empty
362 ;; centre). Any single-end-only configuration (just c1, just c3) is
363 ;; rejected because the block would cantilever and topple. Note:
364 ;; centre-only covers (c2), (c1+c2), (c2+c3), (c1+c2+c3); ends-only
365 ;; covers (c1+c3); the only excluded configurations are (c1 only),
366 ;; (c3 only), and (none).
367 (:action put-long-x
368 :parameters (?b - block ?c1 - cell ?c2 - cell ?c3 - cell)
369 :precondition (and
370   (long ?b) (held-flat-x ?b)
371   (east-of ?c1 ?c2) (east-of ?c2 ?c3)
372   (empty ?c1) (empty ?c2) (empty ?c3)
373   (puttable ?c1) (puttable ?c2) (puttable ?c3)
374   ;; CENTRE c2 supported alone, OR both ENDS c1 and c3 supported.
375   ;; Both branches reference only the unary (supported ?c) -- no
376   ;; (exists) inside the action body -- so no witness params are
377   ;; lifted and put-long-x keeps a single arity.
378   (or
379     (supported ?c2)

```

```

380         (and (supported ?c1) (supported ?c3)))
381     )
382     :effect (and
383         (in ?b ?c1) (in ?b ?c2) (in ?b ?c3)
384         (not (empty ?c1)) (not (empty ?c2)) (not (empty ?c3))
385         (not (held-flat-x ?b)) (gripper-empty)
386     )
387 )
388
389 ;; Put a held flat-y long block at c1, c2, c3 (north-of chain).
390 ;; Same (centre-alone) OR (both-ends) support rule as put-long-x.
391 (:action put-long-y
392     :parameters (?b - block ?c1 - cell ?c2 - cell ?c3 - cell)
393     :precondition (and
394         (long ?b) (held-flat-y ?b)
395         (north-of ?c1 ?c2) (north-of ?c2 ?c3)
396         (empty ?c1) (empty ?c2) (empty ?c3)
397         (puttable ?c1) (puttable ?c2) (puttable ?c3)
398         ;; CENTRE c2 alone, OR both ENDS c1 and c3 (unary supported only).
399         (or
400             (supported ?c2)
401             (and (supported ?c1) (supported ?c3)))
402     )
403     :effect (and
404         (in ?b ?c1) (in ?b ?c2) (in ?b ?c3)
405         (not (empty ?c1)) (not (empty ?c2)) (not (empty ?c3))
406         (not (held-flat-y ?b)) (gripper-empty)
407     )
408 )
409
410 ;; Put a held upright long block at c-low, c-mid, c-high (above chain).
411 ;; Only c-low needs the support check -- c-mid and c-high are supported
412 ;; by the block itself once c-low is anchored. Same single-cell
413 ;; support rule as oblong put-upright.
414 (:action put-long-upright
415     :parameters (?b - block ?c-low - cell ?c-mid - cell ?c-high - cell)
416     :precondition (and
417         (long ?b) (held-upright ?b)
418         (above ?c-low ?c-mid) (above ?c-mid ?c-high)
419         (empty ?c-low) (empty ?c-mid) (empty ?c-high)
420         (puttable ?c-low) (puttable ?c-mid) (puttable ?c-high)
421         (supported ?c-low)
422     )
423     :effect (and
424         (in ?b ?c-low) (in ?b ?c-mid) (in ?b ?c-high)
425         (not (empty ?c-low)) (not (empty ?c-mid)) (not (empty ?c-high))
426         (not (held-upright ?b)) (gripper-empty)
427     )
428 )
429
430 ;; The 6 long-block in-hand transforms are identical to the oblong
431 ;; ones at the executor level (the transform rotates whatever is held);
432 ;; we expose them as separate PDDL actions only so the planner can
433 ;; gate them on (long ?b) and so the action trace is unambiguous.
434
435 (:action make-long-upright-from-x
436     :parameters (?b - block)
437     :precondition (and (long ?b) (held-flat-x ?b))
438     :effect (and (not (held-flat-x ?b)) (held-upright ?b))
439 )
440
441 (:action make-long-upright-from-y
442     :parameters (?b - block)
443     :precondition (and (long ?b) (held-flat-y ?b))
444     :effect (and (not (held-flat-y ?b)) (held-upright ?b))

```

```

445 )
446
447 (:action make-long-flat-x-from-upright
448   :parameters (?b - block)
449   :precondition (and (long ?b) (held-upright ?b))
450   :effect (and (not (held-upright ?b)) (held-flat-x ?b))
451 )
452
453 (:action make-long-flat-y-from-upright
454   :parameters (?b - block)
455   :precondition (and (long ?b) (held-upright ?b))
456   :effect (and (not (held-upright ?b)) (held-flat-y ?b))
457 )
458
459 (:action turn-long-x-to-y
460   :parameters (?b - block)
461   :precondition (and (long ?b) (held-flat-x ?b))
462   :effect (and (not (held-flat-x ?b)) (held-flat-y ?b))
463 )
464
465 (:action turn-long-y-to-x
466   :parameters (?b - block)
467   :precondition (and (long ?b) (held-flat-y ?b))
468   :effect (and (not (held-flat-y ?b)) (held-flat-x ?b))
469 )
470 )

```